Track Reconstruction in ATTPCROOTv2-OpenKF

The UKF and genfit Fitting Pipelines

A User Manual

From simulation / raw HDF5 to fitted kinematics

Active Target Time Projection Chamber (AT-TPC)

Repository: `ATTPCROOTv2-OpenKF` branch `OpenKF-Claude`

June 14, 2026

Contents

1 Introduction

This manual describes how to reconstruct charged-particle tracks in the AT-TPC using **ATTPCROOTv2-OpenKF**, covering the track fitters available in the framework:

- **UKF** — the native *Unscented Kalman Filter* built on the vendored OpenKF library (class `EventFit::AtFitterUKF`). It uses a CATIMA energy-loss model, sigma-point propagation through the magnetic field, and an RTS smoother with back-extrapolation to the reaction vertex.
- **AtGenfitter** — the *clean* GENFIT2 reference-track fitter (class `EventFit::AtGenfitter`, `KalmanFitterRefTrack`). It keeps only the working genfit core (`RKTrackRep` + `AtSpacepointMeasurement`) and shares the validated UKF frame conventions (uniform $z_{\text{lab}} = \texttt{ZPadPlane} - z_{\text{digi}}$, signed B field, highest-z_{digi} vertex). It derives from the *modern* `EventFit::AtFitter` base, so it runs in the maintained `AtFitterTask`, and it can gate which species it fits via an upstream Spyral-PID cut (§7).
- **AtGenfit** (legacy) — the original GENFIT2 fitter (class `AtFITTER::AtGenfit`, run via `AtFitterTaskOld`), with SRIM-style energy-loss tables and built-in merging / reclustering / single-vertex pooling. It is `[[deprecated]]` but retained as a validated fallback.

All three fitters consume the *same* pattern-recognition output (`AtPatternEvent`) and produce fitted tracks (`AtTrackingEvent` / `AtFittedTrack`). They are interchangeable at the level of a single FairRoot task, which makes it easy to fit identical track candidates with different methods and compare.

Note

When to use which. The UKF is faster, native, easy to iterate, and uses CATIMA energy loss — prefer it for quick parameter scans and for setups where its tuning has been validated. `AtGenfitter` is now the **recommended genfit path**: it reaches genfit's analytic-Jacobian resolution while sharing the UKF conventions and adding PID gating; on a1975 `run_0106` it converges on $\approx 95.6\%$ of tracks ($\approx 91\%$ forward) and runs $\sim 15\times$ faster than the legacy `AtGenfit`. Reach for the legacy `AtGenfit` only to reproduce older results. The fitters are best used *together*: reconstruct once, then fit with whichever combination you need and compare.

The document is organized as a pipeline:

```
Simulation OR raw HDF5 → Digitization / Unpacking → PSA (hits) →
[Space-charge corr.] → PRA (track candidates) → UKF / genfit fit →
Kinematics / Ex spectrum
```

Section 2 explains the stages. Section 3 covers the environment. Section 4 walks the *simulation* pipeline end-to-end (`16C_pp` example). Section 5 walks the *experimental* pipeline (`a1975` example), including the two fitters side-by-side. Section 6 is the fitter API reference. Section 7 covers particle identification and the in-fitter PID gate. Section 8 documents the crucial magnetic-field-sign / handedness convention. Sections 9–11 cover the output data model, quick-reference cards, and troubleshooting.

2 Pipeline concepts

Every reconstruction is a chain of FairRoot tasks (`FairTask`) added to a `FairRunAna`. Each task reads one or more ROOT branches and writes another. The stages, in order:

Simulation / Unpacking

For simulation, a Geant4 (`TGeant4`) run transports the beam and reaction products and writes Monte-Carlo points. For experiment, `AtUnpackTask` (with `AtHDFUnpacker`) reads raw GET electronics traces from HDF5 into `AtRawEvent`.

Digitization
(Simulation only.) `AtClusterizeTask` converts MC ionization to electron clusters and `AtPulseTask` produces realistic pad signals (`AtRawEvent`), so simulation and experiment share the same downstream code.

PSA `AtPSAtask` (pulse-shape analysis, typically `AtPSAMax`) turns pad waveforms into 3D hits with charge: branch `AtEventH`.

Space-charge correction
(Experiment, optional.) `AtSpaceChargeCorrectionTask` applies distortion look-up tables and writes `AtEventCorrected`.

Pattern recognition (PRA)
`AtPRAtask` (Hough/triplet RANSAC) or `AtSampleConsensusTask` groups hits into clustered track candidates: branch `AtPatternEvent`.

Fitting
`AtFitterTask` (UKF, multi-hypothesis UKF, or the clean `AtGenfitter`) or the legacy `AtFitterTaskOld` (`AtGenfit`) fits each candidate, producing `AtTrackingEvent` (a collection of `AtFittedTrack`) and optionally `AtFitMetadata`. An `AtPIDTask` may run alongside the fitter to write `AtPIDEvent` (Spyral PID observables); `AtGenfitter` can also compute the same PID internally to gate its input.

Note

The fitter only ever sees `AtPatternEvent`. This is why the fitting step can be *iterated independently*: reconstruct once up to PRA, then refit many times with different fitter settings without re-running the slow stages.

3 Environment and build

3.1 Sourcing the environment

From the repository root, always source the environment before running any macro:

```
cd /path/to/ATTPCROOTv2-OpenKF
source build/config.sh # sets VMCWORKDIR, ROOT, FairRoot, library paths
```

`$VMCWORKDIR` points at the repository root and is used throughout the macros to locate geometry, parameter, and resource files.

3.2 Build dependencies

- **ROOT** (with Eve, Geom, RGL, Gui, Physics, Matrix, MathCore, Tree)
- **FairRoot** — the task framework
- **Eigen3** — required by the vendored OpenKF (UKF)
- **CATIMA** — energy-loss model used by the UKF (auto-fetched if absent)
- **GENFIT2** — required for the genfit fitter (see below)

3.3 Enabling genfit

genfit support is *conditional* at build time. The top-level `CMakeLists.txt` calls `find_package2(PUBLIC GENFIT2)`, and `AtReconstruction/CMakeLists.txt` only compiles `AtGenfit.cxx` and `AtSpacePointMeasurement.cxx` when `GENFIT2_FOUND` is true:

```
if(GENFIT2_FOUND)
  set(SRCS ${SRCS} AtFitter/AtGenfit.cxx AtFitter/AtSpacePointMeasurement.cxx ...)
  set(DEPENDENCIES ${DEPENDENCIES} GENFIT2::genfit2)
endif()
```

Important

To build with genfit you must have GENFIT2 installed and the `GENFIT` environment variable (or CMake variable) pointing at its install prefix — the finder expects `$GENFIT/lib/libgenfit2.so` and `$GENFIT/include/`. If `GENFIT2` is not found, the framework still builds, but the genfit fitter is simply absent and only the UKF is available.

4 Simulation pipeline (16C+p example)

This section reconstructs simulated $^{16}\mathrm{C}(p,p)$ data. All macros live in `macro/Simulation/ATTPC/16C_pp/`. Run them from that directory.

4.1 Stage 1 — Simulation

Macro: `C16_pp_sim.C` `void C16_pp_sim(Int_t nEvents = 100, TString mcEngine = "TGeant4")`

```
root -b -q 'C16_pp_sim(10000)'
```

It runs the Geant4 transport for a $^{16}\mathrm{C}$ beam on a hydrogen target with the (p,p) reaction generator, in a 28.5 kG (2.85 T) axial field. Key setup:

```
// 16C beam
AtTPCIonGenerator *ionGen = new AtTPCIonGenerator("Ion", 6,16,0,1, 0,0,2.297, ...);
// (p,p) two-body reaction, CM 20-90 deg
AtTPC2Body *TwoBody = new AtTPC2Body("TwoBody", ..., 40.0, 20.0, 90.0);
// axial field B_z = 28.5 kG
AtConstField *fMagField = new AtConstField();
fMagField->SetField(0., 0., 28.5);
fMagField->SetFieldRegion(-50,50, -50,50, -10,230);
run->SetField(fMagField);
ATTPC->SetGeometryFileName("ATTPC_H300torr.root");
```

Output: `data/attpcsim.root` (MC truth), `data/attpcpar.root` (parameters), `data/geofile_full.root` (geometry).

4.2 Stage 2 — Digitization + PSA + PRA

Macro: `run_digi_attpc.C` `void run_digi_attpc(int nEvents = 10000, float tCluster = 8.0)`

```
root -b -q 'run_digi_attpc.C(10000, 8.0)'
```

Input: `data/attpcsim.root`. Task chain (in order):

```
AtClusterizeTask *clusterizer = new AtClusterizeTask(); // MC -> e- clusters
clusterizer->SetPersistence(kFALSE);
fRun->AddTask(clusterizer);

AtPulseTask *pulse = new AtPulseTask(std::make_shared<AtPulse>(mapping)); // -> pads
pulse->SetPersistence(kTRUE); pulse->SetSaveMCInfo();
fRun->AddTask(pulse);

auto psa = std::make_unique<AtPSAMax>(); psa->SetThreshold(0); // hits
```

```
AtPSAtask *psaTask = new AtPSAtask(std::move(psa));
psaTask->SetPersistence(kTRUE);
fRun->AddTask(psaTask);

AtPRAtask *praTask = new AtPRAtask(); // track candidates (RANSAC/triplet)
praTask->SetTcluster(tCluster); // time-clustering window (us)
praTask->SetPersistence(kTRUE);
fRun->AddTask(praTask);
```

Parameter file `$VMCWORKDIR/parameters/ATTPC.e20009_sim.par`, pad map `$VMCWORKDIR/scripts/Lookup20150611.xml`.

Output: `data/output_digi.root` with branches `AtEventH` (hits) and `AtPatternEvent` (candidates, ready to fit).

4.3 Stage 3 — UKF fit

Macro: `run_ukf_only.C` `void run_ukf_only(int nEvents = 10000)`

```
root -b -q 'run_ukf_only.C(10000)'
```

Input: `data/output_digi.root`. The fitter task:

```
double charge = 1.602176634e-19; // C
double mass = 938.272; // MeV/c^2 (proton)

auto eloss = std::make_unique<AtTools::AtELossCATIMA>(3.553e-5); // g/cm^3, H2 300 torr
eloss->SetProjectile(1, 1, 1); // Z, A, q
eloss->SetMaterial({ std::make_tuple(1,1,1) }); // hydrogen

auto ukf = std::make_unique<EventFit::AtFitterUKF>(charge, mass, std::move(eloss));
ukf->SetBField(ROOT::Math::XYZVector(0, 0, 2.85)); // +2.85 T : SIMULATION sign
ukf->SetUKFParameters(1e-3, 2.0, 0.0); // alpha, beta, kappa
ukf->SetMeasurementSigma(2.0); // mm
ukf->SetMomentumSigmaFrac(0.5); // loose seed
ukf->SetEnableEnergyStraggling(false);
ukf->SetMinClusters(10);
ukf->SetNIterations(1);
ukf->SetZPadPlane(1000.0); // digi->lab z conversion

AtFitterTask *fitterTask = new AtFitterTask(std::move(ukf));
fitterTask->SetInputBranch("AtPatternEvent");
fitterTask->SetOutputBranch("AtTrackingEvent");
fitterTask->SetFitMetadataBranch("AtFitMetadata");
fitterTask->SetPersistence(kTRUE);
fRun->AddTask(fitterTask);
```

Output: `data/output_ukf_only.root`, branch `AtTrackingEvent` (fitted p, θ, ϕ, KE) plus `AtFitMetadata`.

Note

Validated performance on this channel (∼10k events): efficiency ≈ 95.6%, KE bias −0.2% with RMS ≈ 5%, θ RMS ≈ 0.7°.

4.4 Stage 4 — Display and analysis

```
root -l 'run_ukf_display.C(1)' # interactive: clusters + fit + MC truth (event 1)
root -l 'display_ukf.C(3)' # WSL-safe static projections (XY/XZ/YZ + 3D)
root -b -q 'run_ukf_perf.C(500)' # timing / performance stats
root -b -q show_kinematics.C # KE vs theta, resolution plots
```

4.5 One-shot command sequence

```
cd macro/Simulation/ATTPC/16C_pp/
root -b -q 'C16_pp_sim(10000)' # 1. simulate
root -b -q 'run_digi_attpc.C(10000,8.0)'# 2. digi + PSA + PRA
root -b -q 'run_ukf_only.C(10000)' # 3. UKF fit
root -l 'run_ukf_display.C(1)' # 4. inspect
```

Note

There are also combined macros (`run_digi_ukf.C`, `run_reco_ukf.C`) that fold digitization and fitting into one process. They are convenient but the single-process digi+UKF chain has been observed to hang in some configurations; the staged sequence above is the robust path.

5 Experimental pipeline (a1975 example)

This section reconstructs experimental ^{16}C+p data (experiment a1975) from raw HDF5. Macros are in `macro/Unpack_HDF5/a1975/UKF/`; run them from that folder (the "UKF root"), as data is read by bare name and plots are written to each subfolder's `plots/`. The reusable infrastructure is under `pipeline/`.

5.1 The staged unpacking macros

The pipeline is built incrementally so each stage can be validated and cached. All read raw HDF5 from `/mnt/f/a1975/h5/<run>.h5`.

Macro	Output	Tasks (in order)
`unpack_a1975_UKF.C`	`<run>_unpack.root`	Unpack only
`unpackPSA_a1975_UKF.C`	`<run>_psa.root` / `_disp.root`	Unpack + PSA
`unpackReco_a1975_UKF.C`	`<run>_reco.root`	Unpack + PSA + SC + PRA
`unpackNFit_a1975_UKF.C`	`<run>_ukf_full.root`	Unpack + PSA + SC + PRA + UKF

`unpackReco_a1975_UKF.C` is the **standard entry point**: it produces the `AtPatternEvent` that all three fitters consume. Its task chain:

```
auto unpacker = std::make_unique<AtHDFUnpacker>(fAtMapPtr);
unpacker->SetInputFileName(inputFile.Data());
unpacker->SetNumberTimestamps(2);
unpacker->SetBaseLineSubtraction(true);
auto unpackTask = new AtUnpackTask(std::move(unpacker));
unpackTask->SetPersistence(persistRaw);
fRun->AddTask(unpackTask);

auto psa = new AtPSAMax(); psa->SetThreshold(20);
AtPSAtask *psaTask = new AtPSAtask(psa);
psaTask->SetPersistence(persistRaw);
psaTask->SetOutputBranch("AtEventH");
fRun->AddTask(psaTask);

auto SCModel = std::make_unique<AtEDistortionModel>();
SCModel->SetCorrectionMaps(zlutFile, radlutFile, tralutFile); // a1954 LUTs
auto SCTask = new AtSpaceChargeCorrectionTask(std::move(SCModel));
SCTask->SetInputBranch("AtEventH"); // -> AtEventCorrected
fRun->AddTask(SCTask);

AtPRAtask *praTask = new AtPRAtask();
```

```
praTask->SetInputBranch("AtEventCorrected");
praTask->SetOutputBranch("AtPatternEvent");
praTask->SetClusterRadius(15.0);
praTask->SetClusterDistance(7.5);
praTask->SetPersistence(true);
fRun->AddTask(praTask);
```

Resource files: geometry `ATTPC_H1bar_geomanager.root`, map `ANL2023.xml`, parameters `ATTPC.a1954.par`, space-charge LUTs in `$VMCWORKDIR/resources/corrections/a1954/`.

```
# produce the AtPatternEvent once (persistRaw=true keeps raw for displays)
root -b -q 'pipeline/unpackReco_a1975_UKF.C("run_0116", 1000, true)'
```

5.2 Fitter A — UKF

Macro: `pipeline/fitUKF_a1975.C`. It reads `<run>_reco.root` and writes `<run>_ukf<suffix>.root`. Signature (abbreviated):

```
void fitUKF_a1975(TString fileName="run_0116", Long64_t nEvents=-1,
                  TString particles="proton", Int_t bFieldSign=-1,
                  Double_t bFieldMag=2.85, Double_t gasDensity=9.0e-5,
                  TString outSuffix="", TString ioDir="",
                  Double_t measSigma=2.0, Double_t momSigmaFrac=0.3,
                  Int_t nIter=1, Int_t minClusters=10,
                  TString outDir="", Bool_t refTrack=false,
                  Double_t refInflation=4.0);
```

Each particle name builds an `AtFitterUKF` hypothesis (CATIMA energy loss, mass and charge from a particle table), and they are combined in an `AtFitterUKFMulti` that keeps the best χ^2/ndf per track:

```
auto multi = std::make_unique<EventFit::AtFitterUKFMulti>();
for (each name in particles) {
   auto ukf = MakeHypothesis(name, bFieldSign, bFieldMag, gasDensity,
                             measSigma, momSigmaFrac, nIter, minClusters,
                             refTrack, refInflation);
   multi->AddHypothesis(std::move(ukf), name, /*chargeSign=*/0);
}
AtFitterTask *fitterTask = new AtFitterTask(std::move(multi));
fitterTask->SetInputBranch("AtPatternEvent");
fitterTask->SetOutputBranch("AtTrackingEvent");
fitterTask->SetFitMetadataBranch("AtFitMetadata");
fitterTask->SetPersistence(kTRUE);
```

Inside `MakeHypothesis` the decisive call for experimental data is the **field sign** (see Section 8):

```
ukf->SetBField(ROOT::Math::XYZVector(0, 0, bFieldSign * bFieldMag)); // -1 for exp
```

```
# fast default fit (proton, exp field sign)
root -b -q 'pipeline/fitUKF_a1975.C("run_0116", -1, "proton", -1)'
# resolution-optimized (tighter sigmas), separate output file
root -b -q 'pipeline/fitUKF_a1975.C("run_0116", -1, "proton", -1, 2.85, 9.0e-5, "_opt", "", 0.5,
    0.1, 1, 10)'
```

Note

Resolution tuning. `measSigma=0.5` mm with `momSigmaFrac=0.1` gives noticeably better excitation-energy FWHM than the conservative defaults (`2.0` mm, `0.3`). Use `outSuffix` to avoid clobbering files when scanning parameters.

5.3 Fitter B — AtGenfitter (clean genfit)

Macro: `pipeline/fitGenfitter_a1975.C`. Same input (`<run>_reco.root`), output `<run>_genfitter<suffix>.root` (note the `genfitter`, not `genfit`, stem). This is the **recommended genfit path**. It uses the clean `EventFit::AtGenfitter` on the *modern* `AtFitterTask` — the same wrapper as the UKF — so the two share frame conventions exactly. Signature:

```
void fitGenfitter_a1975(TString fileName="run_0106", Long64_t nEvents=-1,
                        TString ioDir="/mnt/f/a1975/reco/",
                        TString outSuffix="", TString outDir="",
                        Double_t bField=-2.85, Int_t minIter=2, Int_t maxIter=5,
                        TString elossName="deuteron_H2_catima.txt",
                        TString pidGate="pid/deuteron_band.json",
                        Double_t measSigma=4.0, Bool_t matEffects=kFALSE,
                        Int_t pdg=1000010020, Double_t massAmu=2.0135532,
                        Int_t Z=1);
```

The setup is deliberately small — a constructor, two setters, and the optional PID gate:

```
auto fitter = std::make_unique<EventFit::AtGenfitter>(
    bField, pdg, massAmu, Z, elossFile, /*noMatEffects=*/!matEffects,
    minIter, maxIter);
fitter->SetZPadPlane(1000.0);
fitter->SetMeasSigma(measSigma); // per-cluster sigma (mm)
if (pidGate.Length())
   fitter->SetPIDGate(pidGate.Data()); // fit only the gated species

AtFitterTask *fitterTask = new AtFitterTask(std::move(fitter));
fitterTask->SetInputBranch("AtPatternEvent");
fitterTask->SetOutputBranch("AtTrackingEvent");
fitterTask->SetFitMetadataBranch("AtFitMetadata");
fitterTask->SetPersistence(kTRUE);

AtPIDTask *pidTask = new AtPIDTask(); // also persist PID for analysis
pidTask->SetInputBranch("AtPatternEvent");
pidTask->SetOutputBranch("AtPIDEvent");
run->AddTask(pidTask);
run->AddTask(fitterTask);
```

Note the **signed** field default (`bField=-2.85`, the a1975 experimental handedness — §8); no separate "simulation convention" flag exists, the sign *is* the convention. `elossName=""` falls back to genfit's internal Bethe–Bloch. The vertex is taken as the highest-z_{digi} cluster end (matching PRA and the UKF), and a fitted-θ acceptance window (default 10–170°) drops near-beam / backward junk.

```
# (p,d): deuteron hypothesis, deuteron PID gate (defaults)
root -b -q 'pipeline/fitGenfitter_a1975.C("run_0106", -1)'
# (p,p): proton hypothesis + proton gate, separate output stem
root -b -q 'pipeline/fitGenfitter_a1975.C("run_0106", -1, "/mnt/f/a1975/reco/", "_pp", "", -2.85,
     2, 5, "proton_H2_catima.txt", "pid/proton_band.json", 4.0, kFALSE, 2212, 1.00727646, 1)'
```

Note

Why "clean". `AtGenfitter` drops the legacy merging, reclustering, single-vertex pooling and per-direction frame special cases that made `AtGenfit` hard to reason about. Clusters are ordered by the drift coordinate z_{digi} (monotonic along any helix, so curling tracks sequence correctly). On a1975 `run_0106` this gives $\approx 95.6\%$ convergence ($\approx 91\%$ forward) at $\sim 15\times$ the speed of the legacy fitter; the residual non-converging fraction is dominated by PRA "blob" candidates, not fitter failures.

5.4 Fitter C — AtGenfit (legacy genfit)

Macro: `pipeline/fitGenfit_a1975.C`. Same input (`<run>_reco.root`), output `<run>_genfit<suffix>.root`. The original genfit path, kept as a validated fallback for reproducing older results. Signature:

```
void fitGenfit_a1975(TString fileName="run_0106", Long64_t nEvents=-1,
                     TString particle="deuteron", Double_t magneticField=2.85,
                     Double_t gasMediumDensity=0.083147, TString ioDir="",
                     TString outSuffix="", TString outDir="",
                     TString elossName="proton_D2_600torr.txt",
                     Int_t minIter=5, Int_t maxIter=20);
```

It adds an `AtPIDTask` (caches Spyral PID observables) and then the `AtGenfit` fitter wrapped in `AtFitterTaskOld`:

```
std::string elossFile = dir + "/resources/energy_loss/" + elossName;
auto fitter = std::make_unique<AtFITTER::AtGenfit>(
    magneticField, 0.00001, 1000.0, elossFile, gasMediumDensity,
    pdg, minIter, maxIter, /*noMatEffects=*/1);
fitter->SetIonName(particle);
fitter->SetMass(mass); // atomic mass units
fitter->SetAtomicNumber(Z);
fitter->SetNumFitPoints(1.0);
fitter->SetSimulationConvention(0); // experimental data
fitter->EnableMerging(1);
fitter->EnableSingleVertexTrack(1);
fitter->EnableReclustering(1, 15.0, 7.5);

AtFitterTaskOld *fitterTask = new AtFitterTaskOld(std::move(fitter));
fitterTask->SetInputBranch("AtPatternEvent");
fitterTask->SetOutputBranch("AtTrackingEvent");
fitterTask->SetPersistence(kTRUE);
```

```
root -b -q 'pipeline/fitGenfit_a1975.C("run_0106", 200, "deuteron")'
```

5.5 Comparing the two fitters

Because all three read the identical `AtPatternEvent`, any difference in the result is purely the fitter:

```
root -b -q 'pipeline/unpackReco_a1975_UKF.C("run_0106", 500, true)' # once
root -b -q 'pipeline/fitUKF_a1975.C("run_0106", -1, "deuteron", -1, 2.85, 9.0e-5, "", "", 0.5,
    0.1, 1, 10)'
root -b -q 'pipeline/fitGenfitter_a1975.C("run_0106", -1)' # clean genfit
root -b -q 'pipeline/fitGenfit_a1975.C("run_0106", -1, "deuteron")' # legacy genfit
```

	UKF (`AtFitterUKF`)	**AtGenfitter** (clean)	**AtGenfit** (legacy)
Algorithm	Unscented KF + RTS smoother	genfit reference-track KF	genfit reference-track KF
Energy loss	CATIMA (model)	SRIM table, or internal Bethe–Bloch	SRIM table (file)
Iterations	`nIter` (1–2)	`minIter`–`maxIter` (2–5)	`minIter`–`maxIter` (5–20)
Field sign	signed `SetBField`	signed `SetBField` (-2.85 exp)	sim. convention flag
Re-clustering	internal / adaptive	none (drift-z order)	built-in merge / recluster
PID gating	multi-hyp χ^2	`SetPIDGate` (Spyral PID)	—
Task wrapper	`AtFitterTask`	`AtFitterTask`	`AtFitterTaskOld`
Output	track + metadata	track + metadata	track
Status	native, recommended	recommended genfit	deprecated fallback

5.6 Event display

```
# interactive EVE viewer (needs OpenGL/X11; not available under WSL)
root -l 'pipeline/run_eve_UKF.C("run_0116_disp")'
# WSL-safe static PNGs:
root -b -q 'pipeline/event_png_UKF.C("run_0116_disp", -1)' # busiest event
root -b -q 'pipeline/event_png_batch_UKF.C("run_0116_disp", 0, 24)' # contact sheet
```

Important

Under WSL there is no working OpenGL/Eve — interactive 3D viewers crash. Use the static PNG macros (`event_png_UKF.C`, `event_png_batch_UKF.C`) instead. Both require a `<run>_disp.root` produced with `persistRaw=true`.

5.7 Analysis: excitation-energy spectra and the interactive explorer

Once a run is fitted, per-reaction analysis macros turn `AtTrackingEvent` (plus the ion-chamber gate from the FRIB stream) into kinematics and excitation energy. The ^{16}C(p,p) workflow lives in `macro/Unpack_HDF5/a1975/UKF/pp/` and reads the clean-genfit proton output (`<run>_genfitter_pp.root`):

Macro	Role
`cache_pp_run.C`	Flatten one run's fitted proton kinematics into a small ntuple (`ke, theta, vz, chi2ndf, ic, run`) so the explorer never re-reads the slow FRIB files. Drive it over all runs in parallel, then `hadd`.
`explore_pp.C`	Interactive GUI over the cached ntuple: live sliders for beam energy, χ^2/ndf and θ cuts, IC gate, and per-plot binning; draws the Ex spectrum (Gaussian-fit elastic peak), KE-vs-$\theta_{\rm lab}$, Ex-vs-$\theta_{\rm cm}$, and vertex-z-vs-Ex with kinematic-line overlays.
`prod_ex_pp.C`	Batch production over many runs: IC-gated combined Ex spectrum and elastic KE-vs-θ from the genfit protons.
`ex_vtxcorr.C`	Ex with a vertex energy-loss correction: self-calibrates a linear beam-energy profile $E(z)$ (CATIMA ^{16}C dE/dx) by flattening the forward-elastic Ex tilt, then recomputes Ex per event.

```
# cache one run, then explore interactively (p,p)
root -b -q 'pp/cache_pp_run.C("run_0106","/tmp/ppkin_run0106.root")'
root -l 'pp/explore_pp.C("/tmp/pp_kin.root", 1.007825, 16.0147, "16C(p,p)")'
# production Ex over the run list, and the vertex-eloss-corrected Ex
root -b -q 'pp/prod_ex_pp.C()'
root -b -q 'pp/ex_vtxcorr.C("run_0106","/tmp/run_0106_genfitter_pp.root")'
```

Note

The same explorer serves other channels by passing the ejectile/residual masses, e.g. `explore_pp.C("/tmp/pd_kin.root", 2.014102, 15.010599, "16C(p,d)15C")`. On a1975 the (p,p) elastic core sits at $\sigma \approx 0.18$ MeV and is large-angle-dominated — i.e. limited by angular resolution, not beam energy loss, which is why `ex_vtxcorr.C` moves the peak only marginally.

6 Fitter API reference

6.1 `AtFitterTask` (the FairRoot wrapper)

`AtFitterTask` owns any `EventFit::AtFitter` and drives it per event.

```
AtFitterTask(std::unique_ptr<EventFit::AtFitter> fitter); // takes ownership

void SetInputBranch(TString); // default "AtPatternEvent"
void SetOutputBranch(TString); // default "AtTrackingEvent"
void SetFitMetadataBranch(TString); // enable AtFitMetadata persistence
void SetRawEventBranch(TString); // optional, fitter access to AtRawEvent
void SetEventBranch(TString); // optional, fitter access to AtEvent
void SetPersistence(Bool_t = kTRUE);
```

It reads a `TClonesArray` of `AtPatternEvent` and writes `AtTrackingEvent`. (genfit uses the analogous `AtFitterTaskOld`.)

6.2 `EventFit::AtFitterUKF`

Constructor:

```
AtFitterUKF(double charge, // Coulombs (Z * 1.602176634e-19)
            double mass_MeV, // MeV/c^2
            std::unique_ptr<AtTools::AtELossModel> elossModel);
```

Most-used setters (defaults in parentheses):

Method	Meaning
`SetBField(XYZVector)`	Magnetic field, Tesla (default $\{0, 0, 2.85\}$). Sign matters — see §8.
`SetEField(XYZVector)`	Electric field, V/m ($\{0, 0, 0\}$).
`SetUKFParameters(a,b,k)`	Unscented transform α, β, κ ($10^{-3}, 2, 0$).
`SetMeasurementSigma(mm)`	Hit position uncertainty (1.0). Lower ⇒ better resolution.
`SetMomentumSigmaFrac(f)`	Fractional momentum seed uncertainty (0.1).
`SetMinClusters(n)`	Skip tracks with fewer clusters (3).
`SetNIterations(n)`	Filter passes (1); >1 re-seeds with previous result.
`SetEnableEnergyStraggling(b)`	Energy straggling in propagator (false).
`SetMomentumSeed(p_MeV)`	Override Brho seed if >0.
`SetZPadPlane(z)`	Pad-plane z (mm) for digi→lab; $z_{\text{lab}} = z - z_{\text{digi}}$.
`SetAdaptiveClustering(b)`	Re-cluster inside the fit (true); disable for pre-clustered (GNN) input.
`SetUseArcWalk(b)` / `SetArcWalkParams(...)`	kNN arc-ordering clustering for centered vertices.
`SetUseHelixBackExtrap(b)`	Helix (not linear) back-extrapolation to vertex; needed at high curvature.
`SetBackExtrapMaxPath(mm)`	Max back-extrapolation length to vertex (50).
`SetForceVertexOnBeamAxis(b)`	Force vertex to $xy = (0, 0)$.
`SetUseRefTrack(b)` / `SetUseRefTrackWarmStart(b)`	genfit-style re-linearization around the previous smoothed trajectory (needs `nIter`≥ 2).

`AtFitterUKF::Init()` is a no-op (the filter is created lazily on first fit).

6.3 `EventFit::AtFitterUKFMulti` (multi-hypothesis)

Runs several `AtFitterUKF` hypotheses on each track and keeps the best χ^2/ndf:

```
void AddHypothesis(std::unique_ptr<AtFitterUKF> fitter,
                   std::string name, int chargeSign = 0); // 0 = no PRA-sign filter
```

```
void SetChi2Cap(double cap);
std::size_t GetNHypotheses() const;
```

Note

Pass `chargeSign = 0` for experimental data: the PRA charge sign is calibrated on the simulation handedness and is unreliable on experiment until independently validated, so all hypotheses should be evaluated equally.

6.4 `EventFit::AtGenfitter` (clean genfit)

Constructor (all arguments defaulted):

```
AtGenfitter(Double_t bFieldTesla = -2.85, // signed; exp a1975 handedness
           Int_t pdg = 1000010020, // deuteron (2212 p, 1000020040 alpha)
           Double_t massAmu = 2.0135532, // atomic mass units
           Int_t Z = 1,
           std::string eLossFile = "", // SRIM table; "" -> internal Bethe-Bloch
           Bool_t noMatEffects = kTRUE,
           Int_t minIter = 2,
           Int_t maxIter = 5);
```

Setters (defaults in parentheses):

Method	Meaning
`SetParticle(pdg, massAmu, Z)`	Override the particle hypothesis.
`SetBField(tesla)`	Signed field, Tesla (-2.85). Sign = handedness, see §8.
`SetZPadPlane(z)`	Pad-plane z (mm); $z_{\text{lab}} = z - z_{\text{digi}}$ (1000).
`SetMinClusters(n)`	Skip candidates with fewer clusters (4).
`SetIterations(mn, mx)`	Min / max Kalman iterations (2, 5).
`SetMeasSigma(mm)`	Per-cluster measurement resolution (4.0); absorbs scattering when material effects are off.
`SetVertexAxisMaxDist(mm)`	Keep only tracks whose closest cluster is within this of the beam axis (150).
`SetThetaWindow(min, max)`	Fitted-θ acceptance window in deg (10–170); outside ⇒ track dropped.
`SetQualityCut(chi2NdfMax, keMin, keMax)`	Post-fit flag only (5.0, 0.5, 60); failing tracks are kept but `GetGoodFit()==false`.
`SetPIDGate(jsonFile)`	Fit only tracks inside a Spyral-PID gate, see §7.

It uses `genfit::KalmanFitterRefTrack` with `RKTrackRep` and `AtSpacepointMeasurement`, and (like the legacy fitter) requires a loaded `TGeoManager`. Clusters are ordered by drift coordinate z_{digi}; the vertex is the highest-z_{digi} end (lowest z_{lab}), matching PRA and the UKF.

6.5 `AtFITTER::AtGenfit` (legacy)

Constructor:

```
AtGenfit(Float_t magfield, // Tesla (stored internally as 10x)
        Float_t minbrho, // min Brho, Tm
        Float_t maxbrho, // max Brho, Tm
        std::string eLossFile, // SRIM-style energy-loss table
        Float_t gasMediumDensity,// g/cm^3
        Int_t pdg = 2212, // 2212 p, 1000010020 d, 1000020040 alpha
```

```
        Int_t minit = 5, // min Kalman iterations
        Int_t maxit = 20, // max Kalman iterations
        Bool_t noMatEffects = kFALSE);
```

Key setters: `SetIonName`, `SetMass` (amu), `SetAtomicNumber`, `SetPDGCode`, `SetMinIterations` / `SetMaxIterations`, `SetMinBrho` / `SetMaxBrho`, `SetNumFitPoints` (fraction of hits, 0–1), `SetSimulationConvention` (true = simulation θ convention; false = experiment), `EnableMerging`, `EnableSingleVertexTrack`, `EnableReclustering(on, radius, size)`, `SetVerbosityLevel`.

It uses `genfit::KalmanFitterRefTrack` (DAF is available but not instantiated). Measurements come from `AtSpacepointMeasurement`, which converts `AtHitCluster` positions mm→cm and uses a fixed covariance (diagonal $1, 1, 1.28$, the larger z term reflecting drift-time resolution).

Important

`AtGenfit` requires `TGeoManager` (geometry) to be loaded before construction — it caches material properties for all volumes. The class is marked `[[deprecated]]` but remains the production genfit path.

7 Particle identification and the PID gate

Particle identification uses a C++ port of Spyral's algorithm. `AtPIDTask` (class `AtTools::AtSpyralPID` / `AtPIDEstimator`) reads `AtPatternEvent` and writes `AtPIDEvent`, a per-track set of observables — magnetic rigidity $B\rho$, a truncated-mean $\sqrt{dE/dx}$ on the denoised PRA clusters, integrated charge, and cluster count. Plotting $\sqrt{dE/dx}$ versus $B\rho$ separates species into the familiar PID bands.

```
AtPIDTask *pidTask = new AtPIDTask();
pidTask->SetInputBranch("AtPatternEvent");
pidTask->SetOutputBranch("AtPIDEvent");
pidTask->SetPersistence(kTRUE);
run->AddTask(pidTask);
```

A 2D acceptance region is stored as an `AtParticleID` JSON gate (a polygon in the $(\sqrt{dE/dx}, B\rho)$ plane), e.g. `pid/deuteron_band.json` or `pid/proton_band.json`.

7.1 In-fitter gating with `SetPIDGate`

`EventFit::AtGenfitter::SetPIDGate(jsonFile)` makes the fitter compute the *same* Spyral PID internally (it owns its own `AtSpyralPID`, so it needs no `AtPIDEvent` branch) and skip any track whose PID falls outside the gate *before* the expensive genfit fit. This both speeds up production — only the species of interest is fitted — and removes most mis-identified candidates from the output. The gate is purely an input filter; tracks it rejects never appear in `AtTrackingEvent`.

```
fitter->SetPIDGate("pid/deuteron_band.json"); // fit only deuterons
```

Note

Match the gate to the hypothesis: a proton fit (`pdg=2212`) should use `pid/proton_band.json`, a deuteron fit `pid/deuteron_band.json`. If the JSON path does not exist the macro warns and disables gating (every candidate is fitted), so a silently mistyped path simply removes the speed-up rather than crashing.

8 Coordinate conventions and the magnetic-field sign

Important

This is the single most common source of unphysical fits. Read it before fitting *any* experimental data.

Simulation digitization reverses the drift coordinate ($z_{\text{digi}} = z_{\text{padplane}} - z_{\text{MC}}$, in `AtClusterize`), but experimental data does *not* — its z comes straight from the drift time. The two therefore have **opposite helix handedness**. To make the UKF helix match the data, flip the sign of B_z for experimental data:

```
// simulation:
ukf->SetBField({0, 0, +2.85}); // bFieldSign = +1
// experiment:
ukf->SetBField({0, 0, -2.85}); // bFieldSign = -1 (the a1975 default)
```

Empirically (a1975 run_0116):

B_z	median χ^2/ndf	median KE	physical?
-2.85 T (exp sign)	≈ 0.04	≈ 7.9 MeV	yes
$+2.85$ T (sim sign)	≈ 3.09	≈ 152 MeV	no

For the clean `AtGenfitter` the control is identical to the UKF: it takes a **signed** field, so pass `bField=-2.85` for experiment (the default) and `+2.85` for simulation — there is no separate convention flag. The legacy `AtGenfit` instead uses `SetSimulationConvention(false)` for experimental data.

9 Output data model

Branch	Produced by	Contents
`AtRawEvent`	Unpack / Pulse	Raw pad waveforms (ADC vs time bucket).
`AtEventH`	`AtPSAtask`	3D hits with charge after PSA.
`AtEventCorrected`	SC correction	Hits after space-charge distortion correction.
`AtPatternEvent`	`AtPRAtask`	Clustered track candidates (fitter input).
`AtTrackingEvent`	`AtFitterTask`	`AtFittedTrack`: momentum, vertex, kinematics.
`AtFitMetadata`	`AtFitterTask`	χ^2/ndf, iterations, convergence flag, hypothesis.
`AtPIDEvent`	`AtPIDTask`	Spyral PID observables (Brho, $\sqrt{dE/dx}$, ...).

10 Quick-reference command cards

Simulation (^{16}C+p)

```
cd macro/Simulation/ATTPC/16C_pp/
root -b -q 'C16_pp_sim(10000)' # MC truth -> data/attpcsim.root
root -b -q 'run_digi_attpc.C(10000,8.0)' # digi+PSA+PRA -> data/output_digi.root
root -b -q 'run_ukf_only.C(10000)' # UKF fit -> data/output_ukf_only.root
root -l 'run_ukf_display.C(1)' # inspect
```

Experiment (a1975), UKF

```
cd macro/Unpack_HDF5/a1975/UKF/
root -b -q 'pipeline/unpackReco_a1975_UKF.C("run_0116", 1000, true)' # -> run_0116_reco.root
root -b -q 'pipeline/fitUKF_a1975.C("run_0116", -1, "proton", -1)' # -> run_0116_ukf.root
root -b -q 'pipeline/event_png_UKF.C("run_0116_disp", -1)' # static display
```

Experiment (a1975), genfit (clean, recommended)

```
cd macro/Unpack_HDF5/a1975/UKF/
root -b -q 'pipeline/unpackReco_a1975_UKF.C("run_0106", 500, true)' # shared input
root -b -q 'pipeline/fitGenfitter_a1975.C("run_0106", -1)' # -> run_0106_genfitter.root
```

Experiment (a1975), genfit (legacy fallback)

```
cd macro/Unpack_HDF5/a1975/UKF/
root -b -q 'pipeline/fitGenfit_a1975.C("run_0106", -1, "deuteron")' # -> run_0106_genfit.root
```

One-shot experiment (no iteration)

```
root -b -q 'pipeline/unpackNFit_a1975_UKF.C("run_0116", 1000, "proton", -1, 2.85, 9.0e-5)'
```

11 Troubleshooting

Unphysical KE / huge χ^2 on experimental data
Almost always the magnetic-field sign. Use `bFieldSign = -1` (UKF) or `SetSimulationConvention(false)` (genfit). The elastic recoil locus should sit at a few MeV; see §8.

"Input file not found / run unpackReco first"
The fitter macros require `<run>_reco.root`. Run `unpackReco_a1975_UKF.C` once to produce it.

Event display crashes / hangs (WSL)
No OpenGL under WSL. Use `event_png_UKF.C` / `event_png_batch_UKF.C` instead of `run_eve_UKF.C`.

genfit fitter is missing
The build did not find GENFIT2. Set `$GENFIT` and rebuild (§3); without it neither `AtGenfitter` nor `AtGenfit` is compiled and only the UKF is available.

Which genfit fitter to run
Use the clean `AtGenfitter` (`fitGenfitter_a1975.C`) for all new work — it is faster, shares the UKF conventions, and supports PID gating. Fall back to the legacy `AtGenfit` (`fitGenfit_a1975.C`) only to reproduce older results. Mind the output stems: `_genfitter` vs `_genfit`.

PID gate "not found"; everything still fitted
The `pidGate` path passed to `fitGenfitter_a1975.C` does not resolve, so gating is disabled (a warning is printed) and every candidate is fitted. Pass a gate that matches the hypothesis (`pid/proton_band.json` for a proton fit, `pid/deuteron_band.json` for a deuteron fit); see §7.

Poor excitation-energy resolution
Tighten the UKF measurement model: `measSigma`≈ 0.5 mm and `momSigmaFrac`≈ 0.1 instead of the conservative defaults.

Fitter settings that helped one channel hurt another
UKF/PRA knobs are physics-dependent — always re-validate per reaction and per field setting; do not copy tuning blindly between setups.

Source paths in this manual are relative to the ATTPCROOTv2-OpenKF repository root. Code excerpts are condensed for readability; consult the macros and the `AtReconstruction/AtFitter/` headers for exact, current signatures.